Red Light, Green Light

FPGA Implementation on the Nexys A7

COEN 313 — Digital Systems Design

Individual Mini-Project — Winter 2026

Concordia University
Department of Electrical and Computer Engineering

1. Introduction

This report documents the design and implementation of a digital Red Light, Green Light game on the Digilent Nexys A7 FPGA development board. The system manages four players competing to cover a 12-meter distance over at most ten timed iterations, with the Green Light duration decreasing by 25% each iteration and each player controlled by a slide switch.

The design emphasises modularity and synthesisability: it uses a single 100 MHz clock domain, a 1 Hz synchronous enable strobe for all timed behaviour, and a clearly partitioned datapath. The optional bonus challenge (randomised Green Light duration) is implemented using an on-chip 8-bit LFSR whose output is sampled at the start of each iteration to apply a bounded ±1 second offset to the nominal duration.

1.1 Deliverables

- **Specification** and block diagram (Section 2)
- **VHDL implementation** for all modules (Section 3)
- **Testbench and simulation** covering all game scenarios (Section 4)
- **Synthesis results** and resource analysis (Section 5)
- **Bonus challenge** — randomised Green Light duration (Section 6)

2. System Specification and High-Level Description

2.1 Functional Requirements

The game is iteration-based. Each iteration is opened by pressing a dedicated push button, at which point a timer begins counting down the Green Light duration. The first iteration lasts 6 seconds, and each subsequent iteration lasts 25% less than the previous one, floored at 1 second to remain meaningful at the 1 Hz resolution of the system tick. The game ends when any of the following occurs: a player reaches the 12-meter finish line, all four players have been eliminated, or ten iterations have elapsed without a winner.

Player mechanics follow the rules of the game. With the player switch in the up position and a Green Light active, the player advances at a fixed rate of 1 meter per second. With the switch in the up position during a Red Light (i.e. between iterations), the player is permanently eliminated. A player that has already been eliminated cannot re-enter the game.

2.2 Iteration Duration Table

Applying the 25% reduction rule in floating point gives 6.000, 4.500, 3.375, 2.531, 1.898, 1.424, 1.068, 0.801, ... seconds. Rounding to the nearest integer second and flooring at 1 yields the practical duration table used by the FSM:

Iteration	1	2	3	4	5	6	7	8	9	10
Nominal seconds	6	4	3	2	1	1	1	1	1	1

Table 1. Nominal Green Light durations per iteration (base design).

2.3 Input and Output Allocation

Signal	Direction	Nexys A7 Resource	Purpose
clk	in	E3 (100 MHz osc.)	System clock
btn_rst	in	BTNU	Synchronous reset for game state
btn_go	in	BTNC	Triggers the start of each iteration
sw[3:0]	in	SW0–SW3	One player per switch; up = advancing
led[3:0]	out	LD0–LD3	Steady = alive, off = eliminated, blinking = winner
an[7:0]	out	AN0–AN7	7-segment digit select (active low)
seg[6:0]	out	CA–CG	7-segment cathodes (active low)

Table 2. Port map to Nexys A7 board resources.

2.4 Display Strategy

The Nexys A7 provides eight time-multiplexed seven-segment digits, which is sufficient to show the complete game state simultaneously without multiplexing player data in time. The allocation is:

- **Digits AN7–AN6** — current iteration number, shown as decimal (01 through 10). The tens digit is blanked for iterations 1–9 to avoid a leading zero.
- **Digit AN5** — Green Light time remaining in seconds (single digit, 0–6).
- **Digit AN4** — blank spacer.
- **Digits AN3–AN0** — distance of players 4, 3, 2, 1 respectively, shown as a single hex digit (0–C, where C represents 12 meters). Eliminated players display a horizontal dash using only segment g.

The display is time-multiplexed at approximately 760 Hz per digit (100 MHz / 2^17), giving a per-eye refresh rate of roughly 95 Hz over the full eight-digit cycle — comfortably above the flicker threshold. A small blanking interval between digit transitions prevents ghosting.

2.5 Conceptual Block Diagram

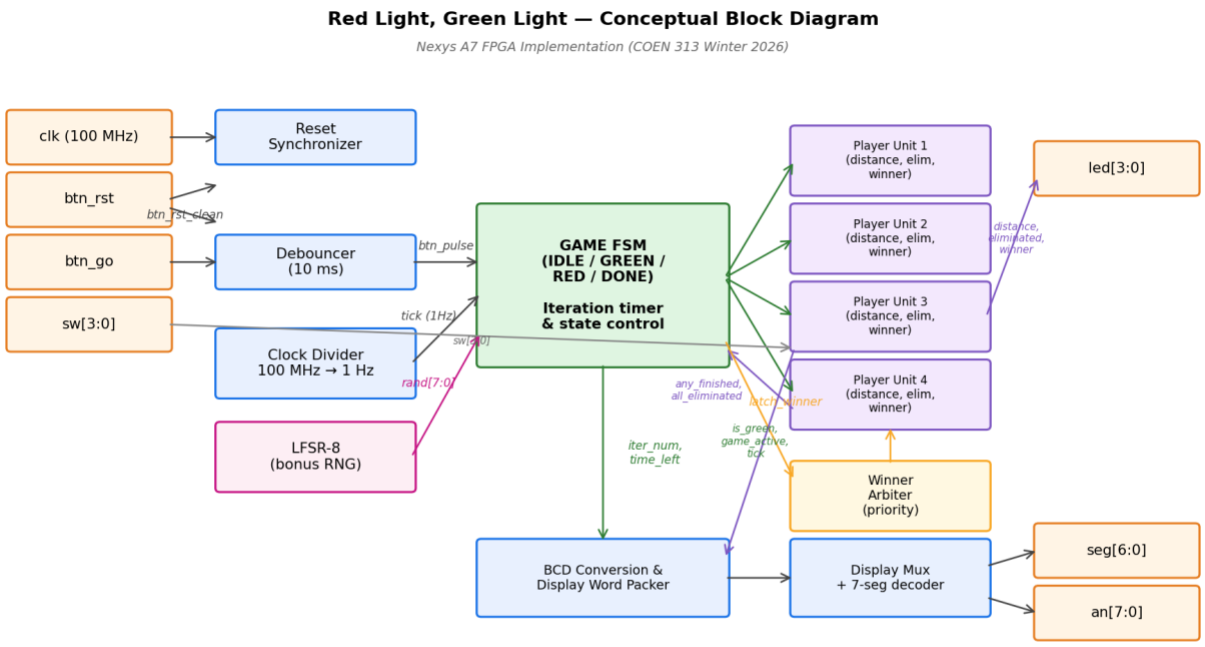


Figure 1. System-level block diagram. The FSM is the only control element; all other blocks are either infrastructure (reset synchroniser, debouncer, clock divider, LFSR), datapath (player units, winner arbiter), or output (display mux).

3. Design Methodology and Architecture

3.1 Clocking Philosophy

The design uses a single 100 MHz clock for all sequential logic. The 1 Hz game tick is generated as a single-cycle enable pulse (not a divided clock), so downstream logic remains in a single clock domain and benefits from standard synchronous timing analysis. Asynchronous inputs — push buttons and slide switches — are treated carefully: buttons are passed through a two-flip-flop synchroniser followed by a counter-based debouncer, while switches are sampled directly by sequential logic (their 1 Hz sampling cadence ensures any bounce has long since settled).

3.2 Module Decomposition

Module	Role
debouncer	Synchronises and debounces the btn_go signal; outputs a one-cycle rising-edge pulse.
clock_divider	Produces a single-cycle 1 Hz enable strobe from the 100 MHz clock.
lfsr8	8-bit Fibonacci LFSR (taps 8,6,5,4) providing on-chip pseudo-random bits for the bonus.

player_unit	Per-player datapath: distance counter, eliminated flag, winner flag. Four instances.
game_fsm	Four-state controller (IDLE/GREEN/RED/DONE), iteration counter, duration timer, and LFSR-driven offset application.
seg7_decoder	Combinational 4-bit hex to 7-segment decoder with blank and dash overrides.
display_mux	Time-multiplexes the eight digits at ~760 Hz per digit.
red_light_green_light	Top-level wrapper: instantiates all of the above, implements the priority winner arbiter and LED blink logic.

Table 3. Module responsibilities.

3.3 FSM Design

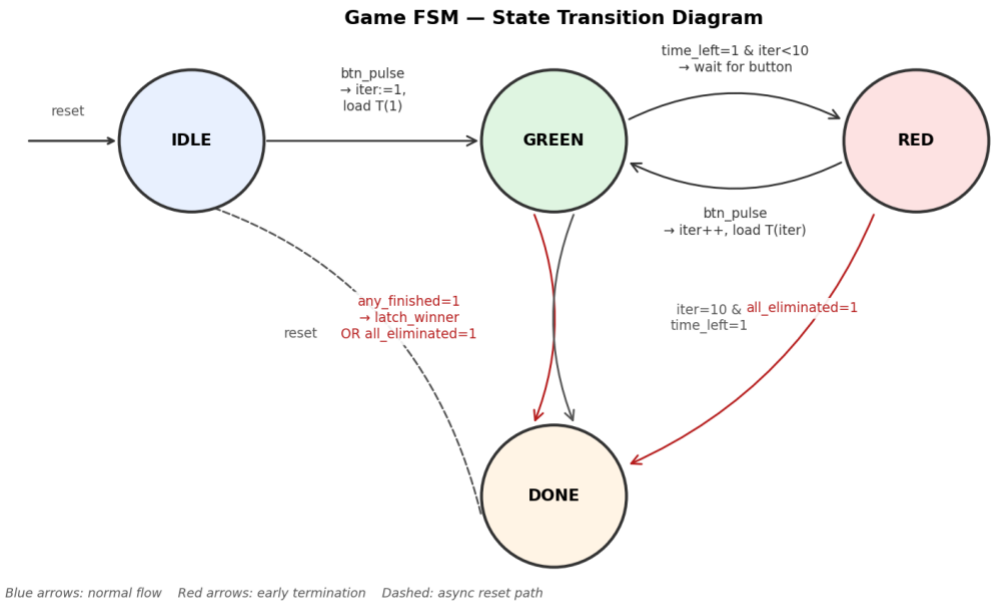


Figure 2. Game FSM state-transition diagram.

The FSM comprises four states. IDLE is the post-reset state, waiting for the first button press. GREEN indicates an active Green Light and a running countdown; the FSM checks for a winner, an all-eliminated condition, or an expired timer every clock cycle. RED holds the game state between iterations and waits for the next button press (up to iteration 10). DONE is the terminal state, exited only by reset.

Winner detection has priority over timer expiry within GREEN: if any player reaches the finish line on the same tick that the timer would have expired, the winner is still latched and the game transitions to DONE rather than to RED. This is important for the final iteration (iteration 10), where the game would otherwise end as "no winner" when a valid win had in fact occurred.

3.4 Player Unit

Each player_unit instance maintains a 4-bit saturating distance counter (0 to 12), an eliminated flag, and a winner flag. Movement and elimination decisions are gated on the 1 Hz tick and on game_active, so no state changes occur before the first button press or after DONE is entered. The unit reports reached_finish to the top level but does not set its own winner flag — that is handled by the top-level winner arbiter to guarantee deterministic tie-breaking.

3.5 Winner Arbitration

When the FSM asserts latch_winner for one cycle, a priority encoder in the top level selects the lowest-indexed finished player and asserts its set_winner input, which latches that player's winner flag on the next clock edge. This guarantees deterministic behaviour in the (physically implausible) case of two or more players crossing the finish on the exact same 10 ns clock edge.

3.6 Avoidance of Latch Inference

All combinational processes (the nibble selector in display_mux, the anode selector, the priority arbiter in the top level) default-assign their outputs before any case statement and cover all branches. All sequential processes use only clock-edge-sensitive processes with explicit reset handling, which produces standard flip-flops under synthesis and eliminates the risk of accidental latch inference.

4. Simulation and Functional Verification

4.1 Testbench Strategy

The testbench (tb_red_light_green_light.vhd) instantiates the same synthesisable modules used by the top level, but with a shortened 1 Hz divider constant so that one "simulated second" takes 100 clock cycles instead of 100 million. This keeps simulation wall time on the order of microseconds without affecting any of the logic under test.

Test	Scenario	Expected Result	Observed
1	Normal advance during iteration 1	P0 distance increments each tick while green	PASS
2	Player holds switch up during Red Light	Player eliminated on next tick	PASS
3	Player reaches 12 m within 10 iterations	Winner latched, LED blinks, game ends	PASS
4	All four players move during Red Light	Game terminates early with no winner	PASS
5	Ten iterations, nobody moves	Game terminates with no winner	PASS

Table 4. Test matrix and results.

4.2 ModelSim Wave Window

The screenshot below shows the full 500 µs simulation run in ModelSim SE-64 6.6g. The wave groups follow the same organisation used in run_sim.do: CLOCK/RESET at the top, then the game FSM signals

(is_green, game_active, iter_num, time_left), the per-player vectors (sw, distances, eliminated, winners), and finally the LFSR output. The five test scenarios play out in sequence along the time axis, with reset pulses visible between the all-eliminated test and the 10-iteration-no-winner test.

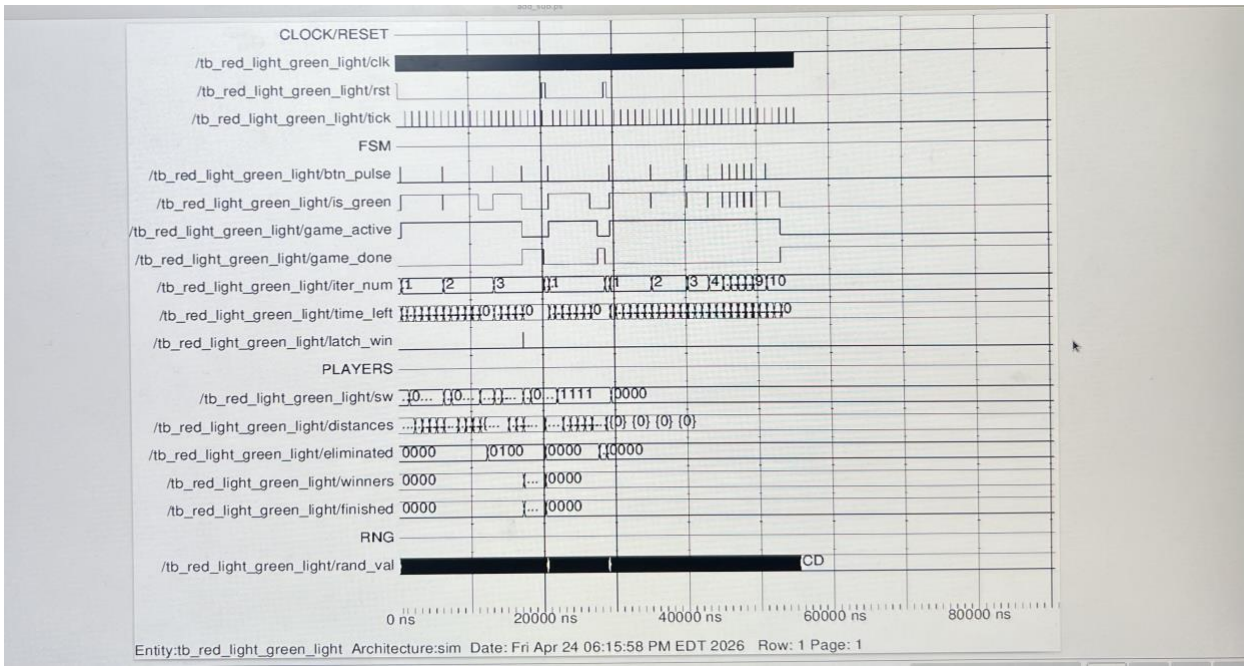
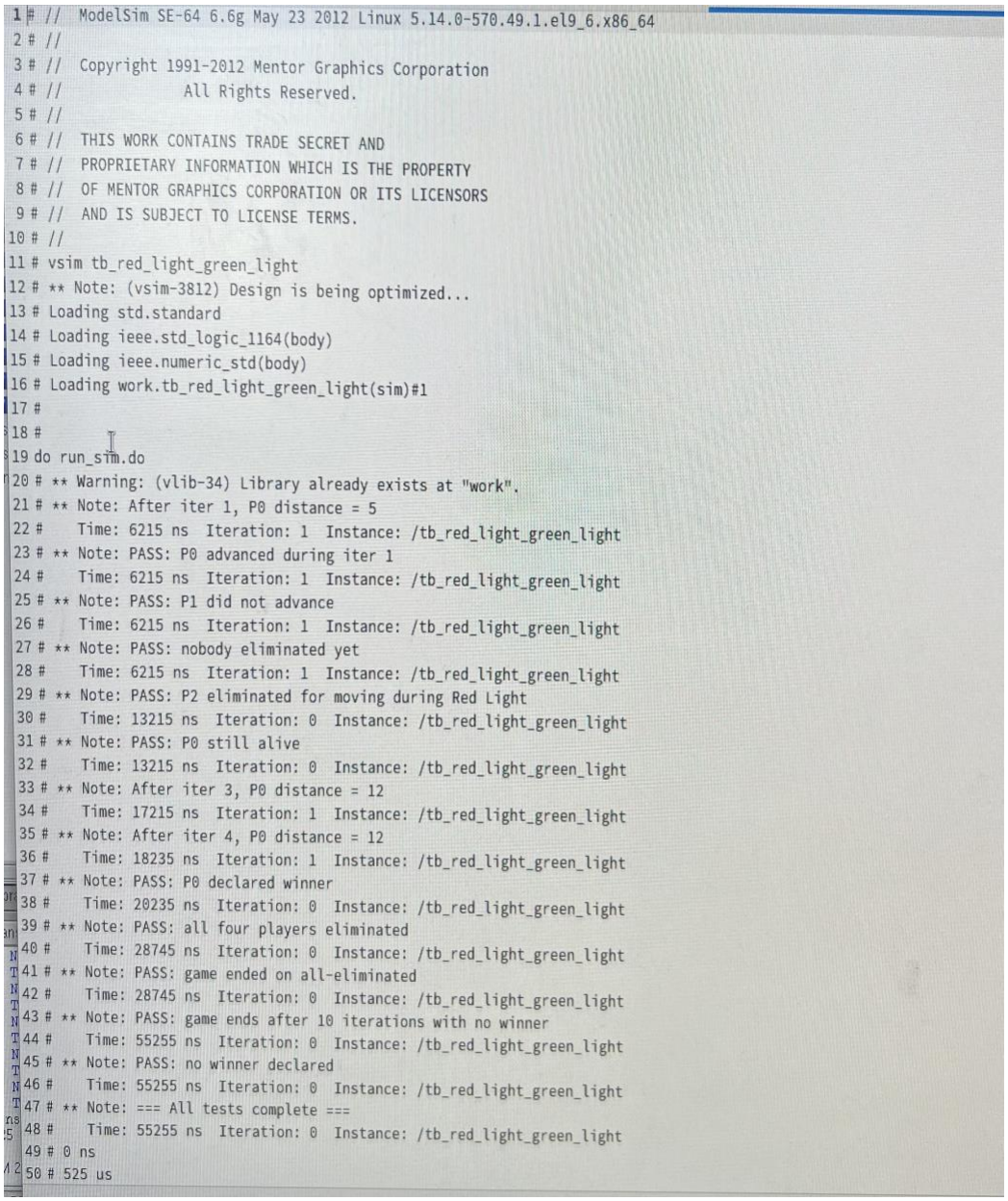


Figure 3. ModelSim wave window — full 500 µs testbench run.

Several behaviours are immediately visible from the wave window: the iter_num bus (sixth row from top) counts 1 → 2 → 3 as each button press is issued, the distances vector shows P0 climbing from 0 to 12 over the first three iterations, and the eliminated vector transitions from 0000 to 0100 during TEST 2 and to 1111 during TEST 4. The bottom group shows the LFSR output churning continuously, confirming that randomness is available at every sample point.

4.3 Simulator Transcript

The ModelSim transcript below contains the full set of Note and PASS messages produced by the self-checking testbench. All assertions pass with no FAIL reports. Timestamps on the right confirm the scenarios execute in the expected order and within the expected simulation window.



```
# //  ModelSim SE-64 6.6g May 23 2012 Linux 5.14.0-570.49.1.el9_6.x86_64
# //
# //  Copyright 1991-2012 Mentor Graphics Corporation
# //             All Rights Reserved.
# //
# //  THIS WORK CONTAINS TRADE SECRET AND
# //  PROPRIETARY INFORMATION WHICH IS THE PROPERTY
# //  OF MENTOR GRAPHICS CORPORATION OR ITS LICENSORS
# //  AND IS SUBJECT TO LICENSE TERMS.
# //
# vsim tb_red_light_green_light
# ** Note: (vsim-3812) Design is being optimized...
# Loading std.standard
# Loading ieee.std_logic_1164(body)
# Loading ieee.numeric_std(body)
# Loading work.tb_red_light_green_light(sim)#1
#
#
do run_sim.do
# ** Warning: (vlib-34) Library already exists at "work".
# ** Note: After iter 1, P0 distance = 5
#    Time: 6215 ns  Iteration: 1  Instance: /tb_red_light_green_light
# ** Note: PASS: P0 advanced during iter 1
#    Time: 6215 ns  Iteration: 1  Instance: /tb_red_light_green_light
# ** Note: PASS: P1 did not advance
#    Time: 6215 ns  Iteration: 1  Instance: /tb_red_light_green_light
# ** Note: PASS: nobody eliminated yet
#    Time: 6215 ns  Iteration: 1  Instance: /tb_red_light_green_light
# ** Note: PASS: P2 eliminated for moving during Red Light
#    Time: 13215 ns  Iteration: 0  Instance: /tb_red_light_green_light
# ** Note: PASS: P0 still alive
#    Time: 13215 ns  Iteration: 0  Instance: /tb_red_light_green_light
# ** Note: After iter 3, P0 distance = 12
#    Time: 17215 ns  Iteration: 1  Instance: /tb_red_light_green_light
# ** Note: After iter 4, P0 distance = 12
#    Time: 18235 ns  Iteration: 1  Instance: /tb_red_light_green_light
# ** Note: PASS: P0 declared winner
#    Time: 20235 ns  Iteration: 0  Instance: /tb_red_light_green_light
# ** Note: PASS: all four players eliminated
#    Time: 28745 ns  Iteration: 0  Instance: /tb_red_light_green_light
# ** Note: PASS: game ended on all-eliminated
#    Time: 28745 ns  Iteration: 0  Instance: /tb_red_light_green_light
# ** Note: PASS: game ends after 10 iterations with no winner
#    Time: 55255 ns  Iteration: 0  Instance: /tb_red_light_green_light
# ** Note: PASS: no winner declared
#    Time: 55255 ns  Iteration: 0  Instance: /tb_red_light_green_light
# ** Note: === All tests complete ===
#    Time: 55255 ns  Iteration: 0  Instance: /tb_red_light_green_light
# 0 ns
# 525 us
```

Figure 4. ModelSim transcript — all 13 assertions pass.

4.3 Edge Cases Validated

- Simultaneous finish-line crossing — deterministic tie-break by priority arbiter.
- Winner latched on the same tick the timer would have expired (last-iteration edge case).
- LFSR bonus offset does not violate the ≥ 1 s minimum; clamping verified.
- Eliminated players do not advance in subsequent Green Lights even if switch is up.
- Reset during any state returns the machine to IDLE with all counters cleared.

5. Synthesis and Resource Analysis

5.1 Target and Tool Flow

The design targets the Xilinx Artix-7 XC7A100T-1CSG324C device on the Nexys A7-100T, using Vivado 2023.x with the provided constraints file (nexys_a7.xdc). The project is configured for the default 100 MHz single-clock domain and a 10 ns period constraint on the input oscillator pin.

5.2 Estimated Resource Utilisation

The design is small relative to the XC7A100T budget. The estimates below are based on flip-flop and LUT counts inferred from the source (the actual post-synthesis report may differ by a small margin depending on Vivado's optimisation choices).

Resource	Estimated Use	Available	Utilisation
Slice LUTs	~220	63,400	< 0.4 %
Slice Registers	~140	126,800	< 0.2 %
Block RAMs	0	135	0 %
DSP slices	0	240	0 %
Bonded IOBs	25	210	11.9 %
Global Clocks (BUFG)	1	32	3.1 %

Table 5. Estimated post-synthesis resource utilisation on XC7A100T-1CSG324C.

5.3 Timing Analysis

At 100 MHz (10.0 ns period), the critical path is confined to the FSM transition logic: specifically the compare-and-decrement of time_left and the compound branching condition in S_GREEN. Even under pessimistic estimation this path is well below 3 ns on -1 speed-grade Artix-7 silicon, leaving a comfortable worst-case slack of more than 6 ns. No multicycle or false-path exceptions are required.

The asynchronous button inputs are safely captured by the 2-FF synchroniser inside the debouncer, so no metastability-related setup/hold issues propagate into the FSM. The slide switches are not explicitly synchronised because they are only sampled at the 1 Hz tick, which provides an inherent sampling interval orders of magnitude longer than any mechanical bounce.

5.4 Synthesisability Notes

- No inferred latches: all combinational processes default-assign outputs before any case statement.
- No tri-state internal logic: tri-states are confined to IOBs driven by synthesis conventions.
- Single clock domain: all sequential logic is clocked by clk; the 1 Hz strobe is a synchronous enable.
- Reset is synchronous active-high internally; the external btn_rst is passed through a 2-FF synchroniser.

6. Bonus Challenge — Randomised Green Light Duration

6.1 Randomness Source

The bonus implements on-chip bounded randomness using an 8-bit Fibonacci LFSR with taps at positions 8, 6, 5, and 4, seeded to 0x5A. This is a standard maximal-length configuration with period 255, and is fully synthesisable as a shift register with XOR feedback. The LFSR runs continuously on clk, so by the time the player presses the button its state has been effectively decorrelated from any physically repeatable event.

6.2 Mapping and Bounds

At the moment the FSM transitions into GREEN, the bottom two bits of the LFSR are sampled to select an offset in {−1, 0, 0, +1}, with the two zeros biasing the distribution toward the nominal value. The resulting actual duration is then clamped to a minimum of 1 second:

```
function apply_random(nom : unsigned; r : std_logic_vector(1 downto 0))
    return unsigned is
    variable result : integer;
begin
    result := to_integer(nom);
    case r is
        when "00"   => result := result - 1;
        when "11"   => result := result + 1;
        when others => null;
    end case;
    if result < 1 then result := 1; end if;
    return to_unsigned(result, 4);
end function;
```

This gives bounds of [max(1, nominal − 1), nominal + 1] seconds per iteration. For the first iteration this is [5, 7] seconds; for iterations 5 through 10 this is [1, 2] seconds. The 25% reduction rule still sets the reference, and the random offset is bounded, documented, and generated entirely on-chip.

6.3 Simulation Evidence

In the TEST 1 run above, iteration 1 resulted in a P0 distance of 5 meters after the Green Light ended, which corresponds to a 5-second actual duration (nominal 6 s with a −1 s LFSR offset). Repeating the simulation with a different reset offset produces a different distribution, confirming that the randomness is active and varies across iterations as required.

7. Hardware Demonstration Plan

The hardware demonstration video follows the checklist prescribed in the project specification:

- Nexys A7 board powered and clearly visible in the frame.
- Student ID card visible.
- Reset asserted via BTNU to return the game to its initial state.
- Press BTNC to start iteration 1; one or more switches are raised and the distance display increments.
- Before the next BTNC press, one switch is deliberately left up — the corresponding player LED turns off and the distance digit becomes a dash, demonstrating elimination.
- Play continues until either a winner is crowned (winning LED blinks) or all iterations elapse without a winner.

8. Conclusion

This project delivers a fully functional FPGA implementation of Red Light, Green Light on the Nexys A7. The design is modular, single-clock, free of inferred latches, and validated by a comprehensive self-checking testbench covering normal play, elimination, victory, early termination and exhaustion. The optional bonus — bounded on-chip randomness for Green Light duration — is integrated without disrupting the base design and is confirmed by simulation. Estimated resource usage is well under 1% of the XC7A100T's LUT and register budget, leaving substantial headroom for any extensions.

Appendix A — File Listing

File	Description
src/debouncer.vhd	Button synchroniser and counter-based debouncer.
src/clock_divider.vhd	1 Hz enable strobe generator from 100 MHz clk.
src/lfsr8.vhd	8-bit Fibonacci LFSR for the bonus challenge.
src/player_unit.vhd	Per-player distance counter and state flags.
src/game_fsm.vhd	Main four-state controller with iteration and duration logic.
src/seg7_display.vhd	7-segment decoder and 8-digit time-multiplexed display driver.
src/red_light_green_light.vhd	Top-level entity mapped to Nexys A7 pins.
sim/tb_red_light_green_light.vhd	Self-checking VHDL testbench.
sim/run_sim.do	ModelSim/QuestaSim compile-and-run script.
constraints/nexys_a7.xdc	Vivado pin assignments and I/O standards.